

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC - KỸ THUẬT MÁY TÍNH



Mạng Máy Tính - CO3094

Đề tài: Implement non-blocking HTTP server and chat application

Giảng viên hướng dẫn: Bùi Xuân Giang
Lớp: L07

Họ và Tên	MSSV
Phạm Đình Phong	2312628
Ngô Đức Thắng	2313184
Đỗ Đăng Khoa	2311581
Lê Hoàng Tân	2313050

Ngày 19 tháng 5 năm 2026

Mục lục

1	Introduction	3
1.1	An Overview	3
1.2	System Goals	3
2	System Architecture	4
2.1	The AsynapRous Framework	4
2.2	Hybrid Chat Paradigm Workflow	4
3	Application and Protocol Design	6
3.1	Server Architecture and Components	6
3.2	Implement Non-blocking Protocol	6
4	HTTP Server with Cookie Session	8
4.1	Authentication Handling Flow	8
4.2	Cookie-based Access Control	8
5	Hybrid Chat Application System Architecture	11
5.1	Overview and Core Modules	11
5.2	Initialization Phase (Client-Server Paradigm)	11
5.2.1	Core Responsibilities of Tracker and State Persistence	12
5.3	Peer-to-Peer Communication Phase	12
5.3.1	Group Communication Security and Client-Side Fan-out	13
5.4	Client Implementation Details	14
5.4.1	Tracker APIs (Centralized Node)	14
5.4.2	Peer APIs (Distributed Node)	14
5.4.3	Frontend Client Integration (<code>chat.html</code>)	15
6	System Validation and Implementation Evidence	16
6.1	AsynapRous Framework Initialization	16
6.1.1	Backend Server Validation	16
6.1.2	Reverse Proxy Server Validation	16
6.1.3	Sample Web Application Validation	17
6.2	Hybrid Chat System Deployment	17
6.3	User Registration and Authentication	18
6.3.1	User Registration (Đăng ký tài khoản)	18



6.3.2	User Login (Đăng nhập)	19
6.4	Hybrid Chat Application: End-to-End P2P Validation	19
6.4.1	Nhắn tin Broadcast trong kênh Global	19
6.4.2	Quản lý và tham gia kênh Chat riêng tư (Private Channel)	20
7	Conclusion	22
7.1	Summary of Achievements	22
7.2	Technical Reflections and Learnings	22
7.3	Closing Thoughts	23

Chương 1

Introduction

1.1 An Overview

Báo cáo này trình bày chi tiết về quá trình nghiên cứu, thiết kế kiến trúc và triển khai thực tế một hệ thống mạng phức hợp cấp thấp, bao gồm máy chủ HTTP không đồng bộ (Non-blocking HTTP Server) và ứng dụng trò chuyện phân tán mô hình lai (Hybrid Chat Application). Hệ thống được phát triển hoàn toàn từ các thư viện Socket nguyên bản của Python, kết hợp chặt chẽ với framework định tuyến tùy chỉnh mang tên **AsynapRous**.

Mục tiêu cốt lõi của đề tài hướng tới việc bóc tách bản chất của các tầng giao vận (Transport Layer) và tầng ứng dụng (Application Layer) thông qua việc kiểm soát dòng luân chuyển byte-stream, xử lý bất đồng bộ, lập trình đa luồng (multi-threading), đồng thời tối ưu hóa bài toán điều phối tài nguyên hệ thống (concurrency) mà hoàn toàn không phụ thuộc vào các framework thương mại bậc cao có sẵn trên thị trường như Express, Flask hay Django.

1.2 System Goals

Để đáp ứng các yêu cầu kiểm thử và tiêu chuẩn kỹ thuật nghiêm ngặt của đồ án môn học, hệ thống được thiết kế để hoàn thành các mục tiêu chiến lược sau:

- **Tối ưu hóa cơ chế I/O tầng thấp:** Triển khai và so sánh toàn diện các mô hình xử lý phi đồng bộ bao gồm Multi-threading truyền thống, Event-driven hướng sự kiện thông qua cơ chế Selector của hệ điều hành, và mô hình Coroutine đa nhiệm dựa trên kiến trúc AsyncIO.
- **Quản lý trạng thái và Bảo mật phiên làm việc:** Thiết kế và triển khai mô hình kiểm soát truy cập (Access Control) tuân thủ nghiêm ngặt các tài liệu đặc tả RFC tiêu chuẩn. Sử dụng HTTP Basic Authentication và Cookie-session để bảo vệ các tài nguyên hệ thống phân tầng.
- **Kiến trúc mạng lai hiệu năng cao:** Xây dựng ứng dụng Chat quy mô lớn, kết hợp hài hòa giữa mô hình Client-Server (Control Plane - quản lý định danh thông tin mạng lưới tại Tracker Server) và mô hình ngang hàng Peer-to-Peer (Data Plane - các node trao đổi gói tin trực tiếp qua TCP Socket), triệt tiêu hoàn toàn hiện tượng nghẽn băng thông tại máy chủ trung tâm.

Chương 2

System Architecture

2.1 The AsynapRous Framework

Kiến trúc hệ thống của framework **AsynapRous** được module hóa sâu sắc để chia tách rạch ròi các trách nhiệm xử lý gói tin và định tuyến, bao gồm 3 thực thể tiến trình (daemon processes) hoạt động độc lập và đồng bộ:

- **Proxy Server Daemon:** Đóng vai trò là điểm chạm kết nối duy nhất (Single Entry Point) từ phía Client. Thành phần này chịu trách nhiệm tiếp nhận luồng Byte thô của HTTP Request, phân tích trường Header Host để phân định Virtual Host, áp dụng chính sách cân bằng tải xoay vòng (Round-robin Load Balancing Strategy) và chuyển tiếp dữ liệu đến Backend thích hợp phía sau.
- **Backend Web Server:** Đơn vị thực thi logic xử lý ứng dụng và ánh xạ tài nguyên thông qua bộ định tuyến Route Decorator tùy chỉnh. Hỗ trợ phản hồi các tệp tin tài nguyên tĩnh (Static Content) và các API RESTful động dưới dạng định dạng dữ liệu mã hóa JSON bytes.
- **Tracker Server Application:** Thành phần đóng vai trò như một thư mục tra cứu phân tán (Directory Service). Tracker giữ nhiệm vụ lưu trữ trạng thái mạng, quản lý danh bạ IP/Port của các Peer đang trực tuyến và thiết lập các chính sách bảo mật cho các phòng chat (Channels).

2.2 Hybrid Chat Paradigm Workflow

Mô hình lai được hiện thực hóa để tối ưu hóa băng thông mạng thông qua hai chu kỳ vận hành tách biệt:

1. **Giai đoạn khởi tạo và cấu hình mạng (Client-Server Phase):** Khi một nút mạng (Peer Node) tham gia vào hệ thống, nó thực hiện một HTTP POST request đến Tracker để đăng ký siêu dữ liệu mạng (Network Metadata) bao gồm địa chỉ Socket hiện tại của nó. Tracker ghi nhận thông tin này vào cấu trúc RAM động và đồng bộ hóa danh bạ Peer Discovery cho toàn bộ các nút khác.

2. **Giai đoạn truyền dữ liệu thời gian thực (Pure P2P Phase):** Khi thực hiện gửi tin nhắn Broadcast hoặc tin nhắn riêng (Private Message), nút nguồn truy vấn danh bạ trực tuyến một lần duy nhất từ Tracker. Sau đó, luồng dữ liệu chuyển hẳn sang Data Plane ngang hàng; nút nguồn chủ động tạo các TCP Socket Client kết nối trực tiếp đến các port lắng nghe của các nút đích để truyền tải tin nhắn, giải phóng hoàn toàn Tracker khỏi nhiệm vụ xử lý relay gói tin.

Chương 3

Application and Protocol Design

Phần này trình bày chi tiết về kiến trúc nền tảng của hệ thống máy chủ HTTP. Theo yêu cầu bắt buộc, framework HTTP phải được xây dựng hoàn toàn thủ công (built by yourself) dựa trên các giao thức không chặn (non-blocking protocol).

3.1 Server Architecture and Components

Hệ thống xử lý các yêu cầu HTTP thông qua một kiến trúc tùy chỉnh được đóng gói trong framework **AsynapRous**. Quá trình xử lý luồng dữ liệu phụ thuộc vào các thành phần cốt lõi sau:

- **daemon.request (Request Class):** Đảm nhiệm việc phân tích (parse) chuỗi byte HTTP thô từ socket để trích xuất phương thức (GET, POST, PUT), đường dẫn (path), trích xuất thông tin Header (bao gồm Authorization) và bóc tách dữ liệu Cookie qua hàm `prepare_cookies`.
- **daemon.response (Response Class):** Xây dựng gói tin HTTP trả về chuẩn xác. Module này tự động xác định MIME type (`text/html`, `application/json`), mã trạng thái (200, 401, 404), và đặc biệt là quản lý các header an ninh như `Set-Cookie`.
- **daemon.httpadapter (HttpAdapter Class):** Đóng vai trò là cầu nối (Bridge) giữa Socket tầng mạng và RESTful API tầng ứng dụng. Nó nhận dữ liệu từ Client, gọi Request để parse, kiểm tra Route, và gọi Response để phản hồi.

Ngoài ra, kiến trúc này cho phép các ứng dụng tầng trên (như Tracker Server và Peer App) dễ dàng đăng ký các hàm xử lý nghiệp vụ thông qua cơ chế Decorator `@app.route`, giúp phân tách hoàn toàn giữa logic xử lý giao thức mạng tầng thấp và logic ứng dụng tầng cao.

3.2 Implement Non-blocking Protocol

Để tránh tình trạng bottleneck khi nhiều Client kết nối cùng lúc, hệ thống sử dụng tham số `mode_async` trong `backend.py` để định tuyến cơ chế I/O. Luồng thực thi được thiết kế để hỗ trợ đa cơ chế:

1. **Cơ chế Event-Driven (Callback):** Sử dụng thư viện `selectors`. Socket của máy chủ được đưa về trạng thái `setblocking(False)`. Hệ thống chỉ gọi hàm xử lý `handle_client_callback` khi có tín hiệu `EVENT_READ` từ hệ điều hành.
2. **Cơ chế Multi-threading:** Đóng vai trò fallback, mỗi khi máy chủ `accept()` một kết nối mới, một `threading.Thread` độc lập sẽ được sinh ra để duy trì kết nối biệt lập.
3. **Cơ chế Coroutine:** Khởi chạy một event loop `asyncio.run(async_server)` chuyên biệt cho các thao tác `async/await` hiệu suất cao.

```
1 def run_backend(ip, port, routes):
2     global mode_async
3
4     # Process async stream using coroutines (asyncio)
5     if mode_async == "coroutine":
6         asyncio.run(async_server(ip, port, routes))
7         return
8
9     # Process socket object for Callback or Threading
10    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
11    try:
12        server.bind((ip, port))
13        server.listen(50)
14
15        # Register server socket with the event selector for read events
16        if mode_async == "callback":
17            sel.register(server, selectors.EVENT_READ, (handle_client_callback,
18                ip, port, routes))
19
20        while True:
21            # Accept incoming connection
22            conn, addr = server.accept()
23
24            # @bksysnet: Dynamic selection mechanism based on mode_async
25            variable
26            if mode_async == "callback":
27                server.setblocking(False) # Strict non-blocking mode
28                events = sel.select(timeout=None)
29                for key, mask in events:
30                    callback, ip, port, routes = key.data
31                    callback(key.fileobj, ip, port, conn, addr, routes)
32            else:
33                # Baseline multi-thread implementation
34                threading.Thread(target=handle_client, args=(ip, port, conn,
35                    addr, routes)).start()
36
37    except socket.error as e:
38        print("Socket error: {}".format(e))
```

Listing 3.1: Dynamic Non-blocking Mechanism Implementation (backend.py)

Chương 4

HTTP Server with Cookie Session

Phần này mô tả chi tiết việc triển khai cơ chế xử lý phiên làm việc dựa trên Cookie để quản lý xác thực người dùng, hoàn thành Task Authentication Handling và Access Control.

4.1 Authentication Handling Flow

Khi hệ thống nhận một truy cập vào các tài nguyên được bảo vệ, `Response Class` sẽ thực thi luồng xác thực (Authentication Flow) như sau:

1. **Credential Extraction:** Phương thức `get_auth_credentials()` trong lớp `Request` bóc tách header `Authorization`. Hệ thống tiến hành giải mã Base64 chuỗi nhận được để tách riêng `username` và `password`.
2. **Credential Verification:** Trong hàm `build_response_header()`, chuỗi xác thực được so khớp tính với cấu hình hệ thống (VD: `Basic YWRtaW46MTIz` tương đương `admin:123`).
3. **Successful Authentication (Session Creation):** Nếu thông tin hợp lệ, máy chủ sinh ra một phiên làm việc thông qua header `Set-Cookie: sessionid=BK_THANG_2026; Path=/; HttpOnly`. Thuộc tính `HttpOnly` giúp ngăn chặn các cuộc tấn công XSS lấy cắp token từ phía Client.
4. **Invalid Authentication (Error Handling):** Nếu sai thông tin hoặc thiếu Session, máy chủ lập tức từ chối phục vụ, trả về mã trạng thái **401 Unauthorized** kèm theo header `WWW-Authenticate` để kích hoạt popup đăng nhập tích hợp trên trình duyệt.

4.2 Cookie-based Access Control

Đối với các tài nguyên đã được bảo vệ, hệ thống triển khai bộ kiểm soát truy cập (Access Control) nghiêm ngặt:

- **Header Inspection:** Khi nhận request, lớp `Request` chạy hàm `prepare_cookies()` để phân rã chuỗi Cookie thô (ngăn cách bởi dấu chấm phẩy) thành một Python Dictionary (VD: `{'sessionid': 'BK_THANG_2026'}`).

- **Token Validation:** Tại hàm `build_response_header()`, hệ thống lấy biến `user_session` và so sánh. Nếu Cookie hợp lệ, luồng thực thi trả về 200 OK và cấp quyền tải file tĩnh (HTML/CSS).

Bên cạnh đó, ở cấp độ ứng dụng distributed ô ngang hàng (`peer_app.py`), hệ thống đã tích hợp thêm một cơ chế đăng ký tài khoản cục bộ lưu trữ lâu dài (Persistent Account Registration) qua endpoint `/register`. Dữ liệu tài khoản của người dùng được phân tách và lưu trữ bền vững dưới dạng cấu trúc tệp tin JSON (`json/users.json`), đảm bảo an toàn định danh trước khi đồng bộ hóa trạng thái phiên lên giao diện Web.

```
1 def build_response_header(self, request):
2     reqhdr = request.headers
3     status_line = "HTTP/1.1 200 OK\r\n"
4
5     # Define valid credentials (admin:123 encoded in Base64)
6     valid_credentials = "Basic YWRtaW46MTIz"
7     auth_header = reqhdr.get("authorization", "")
8     user_session = request.cookies.get('sessionid') if request.cookies else
9     None
10    target_session = "BK_THANG_2026"
11
12    # Route bypass policy
13    is_public_page = request.path == '/register'
14
15    # 1. Reject unauthorized access and prompt login (Task 1A Error / Task 1B
16    Missing Cookie)
17    if not is_public_page and user_session != target_session and auth_header !=
18    valid_credentials:
19        status_line = "HTTP/1.1 401 Unauthorized\r\n"
20        self.headers["WWW-Authenticate"] = 'Basic realm="BK SmartHome Access"'
21        self._content = b"<html><body><h1>401 Unauthorized</h1><p>Vui long dang
22        nhap.</p></body></html>"
23
24    # 2. Grant session via Set-Cookie if login credentials are correct (Task 1A
25    Success)
26    elif auth_header == valid_credentials:
27        self.headers["Set-Cookie"] = "sessionid={}; Path=/; HttpOnly".format(
28        target_session)
29
30    # 3. Construct HTTP Header Bytes
31    headers_dict = {
32        "Date": datetime.datetime.utcnow().strftime("%a, %d %b %Y %H:%M:%S GMT"
33        ),
34        "Server": "AsynapRous-BK-Core",
35        "Content-Type": self.headers.get('Content-Type', 'text/html'),
36        "Content-Length": str(len(self._content)),
37        "Connection": "keep-alive",
38    }
39
40    # Append security headers if present
41    if "WWW-Authenticate" in self.headers: headers_dict["WWW-Authenticate"] =
42    self.headers["WWW-Authenticate"]
```

```
35     if "Set-Cookie" in self.headers: headers_dict["Set-Cookie"] = self.headers["Set-Cookie"]
36
37     fmt_header = status_line + ".join([f"{k}: {v}\r\n" for k, v in
headers_dict.items()]) + "\r\n"
38     return fmt_header.encode('utf-8')
```

Listing 4.1: Authentication Handling and Cookie verification logic (`response.py`)

Chương 5

Hybrid Chat Application System Architecture

Ứng dụng trò chuyện sử dụng kiến trúc mạng lai (Hybrid network architecture), tận dụng độ tin cậy của mô hình Client-Server cho việc khám phá cấu hình mạng (Peer Discovery), quản lý trạng thái, và hiệu năng tối ưu của mô hình Peer-to-Peer (P2P) để trao đổi tin nhắn trực tiếp không qua trung gian.

5.1 Overview and Core Modules

Hệ thống hoạt động dựa trên hai tiến trình song song, được xây dựng bằng các hàm Decorator `@app.route` của framework **AsynapRous**:

1. **Tracker Server (tracker_app.py)**: Một Server độc lập chạy tại cổng 9000, đóng vai trò như một thư mục quản lý tập trung và danh bạ định tuyến. Nó lưu trữ thông tin IP, Port của các Peer đang trực tuyến, danh sách các Kênh trò chuyện hiện có. Tracker triển khai lưu trữ bền vững (Data Persistence) qua tệp tin `json/channels.json`, loại bỏ hoàn toàn rủi ro mất mát dữ liệu phòng chat khi máy chủ khởi động lại. Máy chủ tuyệt đối không lưu trữ nội dung tin nhắn chat để giảm tải băng thông và bảo vệ tính riêng tư.
2. **Peer Application (peer_app.py)**: Các máy khách phân tán chạy độc lập tại cổng 8001, 8002,... vừa đóng vai trò là Client (gửi truy vấn đăng ký/khám phá tới Tracker), vừa làm một Server cục bộ (mở cổng Socket để trực tiếp tiếp nhận tin nhắn từ các Peer khác). Peer sở hữu cơ chế quản lý tài khoản người dùng biệt lập thông qua tệp tin cơ sở dữ liệu `json/users.json`.

5.2 Initialization Phase (Client-Server Paradigm)

Giai đoạn khởi tạo thiết lập cấu trúc mạng lưới (network topology) và đồng bộ cấu hình thông qua Tracker Server.

5.2.1 Core Responsibilities of Tracker and State Persistence

Tracker quản lý toàn bộ tài nguyên hệ thống thông qua các hàm xử lý tệp tin chuyên biệt `load_channels()` và `save_channels()`, cung cấp các dịch vụ endpoint cốt lõi:

- **Unified Registration** (`/submit-info`): Khi khởi động, các Peer tự động gửi payload chứa `username`, `ip`, và `port` của mình lên để Tracker ghi nhận vào từ điển `active_peers`.
- **Channel Lifecycle Management** (`/add-list`, `/delete-channel`): Hỗ trợ khởi tạo phòng trò chuyện mới kèm cơ chế bảo mật mật khẩu. Hàm `/delete-channel` cho phép xóa bỏ hoàn toàn phòng chat khỏi hệ thống lưu trữ tĩnh.
- **Access Control Checking** (`/join-channel`, `/leave-channel`): Quản lý nghiêm ngặt danh sách thành viên trong phòng. Đối với các phòng có thuộc tính bảo mật (`password` khác rỗng), chỉ khi người dùng gửi chính xác chuỗi mã khóa xác thực, Tracker mới cập nhật tên của Peer vào mảng `members` của kênh đó trong file JSON tĩnh. Hàm `/leave-channel` đảm nhiệm việc thu hồi quyền truy cập khi người dùng rời phòng.
- **Peer Discovery** (`/get-list`, `/unregister`): Cung cấp topo mạng thời gian thực cho các Peer dưới dạng JSON (các mật khẩu phòng được che dấu). Khi một Peer đóng kết nối hoặc đăng xuất, API `/unregister` sẽ giải phóng định danh của Peer khỏi danh mục `active_peers`.

```
1 @app.route('/join-channel', methods=['POST'])
2 def join_channel(headers, body):
3     """API for Access Control checking on Tracker with JSON persistence"""
4     data = json.loads(body)
5     u, name, pwd = data.get("username"), data.get("channel_name"), data.get("password", "")
6
7     if name in active_channels:
8         ch = active_channels[name]
9         # Verify access policy: Public channel OR Password matches OR User
10        already a member
11        if u in ch["members"] or ch["password"] == "" or ch["password"] == pwd:
12            if u not in ch["members"]:
13                ch["members"].append(u)
14                save_channels(active_channels) # Persist to json/channels.json
15            return {"status": "success"}
16        return {"status": "error", "message": "Invalid password!"}
17    return {"status": "error", "message": "Channel not found!"}
```

Listing 5.1: Access Control and Persistence Layer on Tracker (`tracker_app.py`)

5.3 Peer-to-Peer Communication Phase

Sau khi hoàn tất giai đoạn khám phá và tải danh bạ từ Tracker thông qua giao thức Client-Server, luồng dữ liệu truyền tải thông điệp chat được chuyển giao hoàn toàn sang kiến trúc mạng ngang hàng P2P thuần túy nhằm tối ưu hóa tài nguyên mạng.

5.3.1 Group Communication Security and Client-Side Fan-out

Để hỗ trợ trò chuyện nhóm (Group Chat) mà không phụ thuộc vào một máy chủ trung tâm làm trạm trung chuyển gói tin (Relay Server), hệ thống áp dụng thuật toán **"Client-Side Fan-out"** kết hợp với bộ lọc chính sách riêng tư bảo mật:

- 1. Membership & Policy Retrieval:** Khi người dùng kích hoạt gửi tin nhắn nhóm, tiến trình Peer tự động gửi một yêu cầu đồng bộ hóa đến Tracker (`/get-list`) nhằm thu thập danh sách ánh xạ các Peer hiện đang hoạt động và siêu dữ liệu cấu hình phòng chat (channels metadata).
- 2. Privacy Filtering & Fan-out Dissemination:** Hệ thống lặp qua toàn bộ danh sách các nút mạng tìm thấy. Tại đây, một bộ lọc kép được áp dụng nghiêm ngặt: loại bỏ chính mình khỏi vòng lặp (`name != my_username`) và thực hiện kiểm tra quyền truy cập nếu kênh ở trạng thái riêng tư (`not is_private or name in members`). Nếu thỏa mãn điều kiện bảo mật, tiến trình sử dụng module `urllib.request` phát trực tiếp gói tin HTTP POST chứa thông điệp chat thuần túy tới địa chỉ IP và Port đích danh của từng Peer đích.

```
1 @app.route('/send-broadcast', methods=['POST'])
2 def api_send_broadcast(headers, body):
3     data = json.loads(body)
4     msg, channel = data.get("message"), data.get("channel", "Global")
5     try:
6         # 1. Topology Discovery and Security Policy Extraction
7         with urllib.request.urlopen("http://127.0.0.1:9000/get-list") as resp:
8             t_data = json.loads(resp.read().decode())
9             online_peers = t_data["peers"]
10            chan_info = t_data["channels"].get(channel, {})
11            members = chan_info.get("members", [])
12            is_private = chan_info.get("password", "") != ""
13
14            # 2. Secured Client-Side Fan-out Multi-cast
15            for name, info in online_peers.items():
16                if name != my_username and (not is_private or name in members):
17                    p2p_url = f"http://{info['ip']}:{info['port']}/broadcast-peer"
18                    p_data = json.dumps({"sender": my_username, "message": msg, "channel": channel}).encode('utf-8')
19
20                    # Asynchronous-like direct socket invocation abstracted via
21                    urllib
22                    urllib.request.urlopen(urllib.request.Request(p2p_url, data=p_data, method='POST'))
23
24                    # Persist locally inside memory array for Web UI rendering
25                    message_history.append({"sender": my_username, "message": msg, "type": "broadcast", "channel": channel})
26                    return {"status": "success"}
27            except Exception as e:
28                return {"status": "error", "message": str(e)}
```

Listing 5.2: Implementation of Secured P2P Client-Side Fan-out (peer_app.py)

5.4 Client Implementation Details

Để hiện thực hóa kiến trúc Web-Client Integration (sử dụng trình duyệt làm giao diện đồ họa), tiến trình cục bộ `peer_app.py` đóng vai trò như một **Local Web Proxy**. Giao diện Frontend HTML/JS chỉ kết nối nội bộ tới môi trường máy chủ cục bộ `localhost`, loại bỏ hoàn toàn các rào cản về CORS và đơn giản hóa mô hình lập trình mạng.

5.4.1 Tracker APIs (Centralized Node)

Bảng 5.1 trình bày chi tiết toàn bộ hệ thống các Route Handler được định nghĩa và đăng ký trên `tracker_app.py` để quản lý kiến trúc mạng tổng thể.

Bảng 5.1: Tracker Server Route Handlers (`tracker_app.py`)

API Endpoint	Responsibility
<code>/submit-info</code>	Registration: Cập nhật thông tin cấu hình định danh mạng (<code>username</code> , <code>ip</code> , <code>port</code>) của một Peer vào danh mục hoạt động.
<code>/unregister</code>	Deregistration: Loại bỏ Peer ra khỏi bộ nhớ hệ thống khi ngắt kết nối hoặc thoát ứng dụng.
<code>/add-list</code>	Room Creation: Khởi tạo phòng chat mới kèm bộ lọc mật khẩu, đồng thời tự động lưu trữ lâu dài vào cơ sở dữ liệu tệp tin JSON.
<code>/delete-channel</code>	Room Deletion: Hủy phòng và giải phóng bộ nhớ lưu trữ tương ứng trên Tracker.
<code>/join-channel</code>	Access Control: Kiểm tra tính hợp lệ của mật khẩu truy cập và đưa Peer vào mảng thành viên được cấp quyền.
<code>/leave-channel</code>	Membership Revocation: Cho phép người dùng chủ động thoát khỏi danh sách thành viên của phòng chat.
<code>/get-list</code>	Discovery Proxy: Xuất bản topo mạng toàn cục (Danh sách Peer và Channel đang hoạt động công khai).

5.4.2 Peer APIs (Distributed Node)

Bảng 5.2 phân tách rõ ràng các API của ứng dụng Peer thành 2 phân hệ chuyên biệt: Nhóm giao tiếp nội bộ phục vụ hiển thị Web UI (Local Proxy Engine) và Nhóm lắng nghe xử lý mạng (P2P Inbound Handlers).

Bảng 5.2: Peer Application Route Handlers (`peer_app.py`)

Category	API Endpoint	Responsibility
Web Bridge	<code>/register</code>	Tiếp nhận thông tin đăng ký tài khoản từ giao diện và lưu trữ xuống file hệ thống <code>users.json</code> .
Web Bridge	<code>/login</code>	Xác thực và thiết lập tên định danh <code>my_username</code> ở bộ nhớ cache local của tiến trình.
Web Bridge	<code>/get-messages</code>	Polling Target: Trả về mảng <code>message_history</code> cục bộ cho giao diện Frontend kết xuất đồ họa lên màn hình.
Web Bridge	<code>/send-broadcast</code>	Outbound Engine: Thực hiện thuật toán kiểm tra chính sách bảo mật riêng tư và kích hoạt Client-Side Fan-out.
Web Bridge	<code>/get-channels</code>	Discovery Gateway: Thay mặt Frontend gọi tới Tracker để lấy dữ liệu phòng chat hiển thị lên sidebar.
P2P Inbound	<code>/connect-peer</code>	Handshake Handler: Tiếp nhận tín hiệu kết nối và soát trạng thái từ các nút mạng ngang hàng khác.
P2P Inbound	<code>/send-peer</code>	Private Inbox: Tiếp nhận thông điệp chat riêng tư và đẩy trực tiếp vào mảng lưu trữ cục bộ dưới danh mục "Private".
P2P Inbound	<code>/broadcast-peer</code>	Group Inbox: Lắng nghe gói tin đa hướng từ luồng Fan-out của các nút khác và phân loại chính xác theo định danh phòng <code>channel</code> .

5.4.3 Frontend Client Integration (`chat.html`)

Giao diện người dùng được xây dựng hoàn chỉnh bằng công nghệ HTML5, CSS biến số tối ưu giao diện tối (Dark mode) và ngôn ngữ JavaScript bất đồng bộ thuần túy nhằm tương tác mượt mà với Local Web Proxy:

- **Cơ chế Short Polling định kỳ (`sync()`):** Frontend triển khai một bộ lập lịch tự động gọi hàm `sync()` liên tục theo chu kỳ thời gian ngắn. Hàm này thực hiện các yêu cầu `fetch()` song song đến `/get-messages` và `/get-channels` của Peer local để đảm bảo giao diện luôn đồng bộ với các tin nhắn mạng P2P mới nhất mà không gây treo luồng dựng giao diện (UI Blocking).
- **Hộp thoại xác thực quyền phòng riêng tư (`prompt`):** Khi người dùng nhấn chọn một phòng chat từ danh sách, ứng dụng Client kiểm tra thuộc tính `is_private` của phòng. Nếu phòng yêu cầu mã khóa bảo mật, ứng dụng sẽ kích hoạt một cửa sổ hội thoại trực quan `prompt()` yêu cầu nhập mật khẩu bảo vệ trước khi phát lệnh POST thực tế tới API `/join-channel` của Tracker Server.
- **Trải nghiệm người dùng tương tác cao (Key Bindings):** Chức năng gửi tin nhắn `sendMessage()` không chỉ liên kết với nút bấm đồ họa mà còn được tích hợp trực tiếp vào trình lắng nghe sự kiện bàn phím (`addEventListener("keypress")`). Hệ thống tự động bắt mã phím "Enter" để tối ưu hóa tốc độ trao đổi thông tin của người sử dụng.

Chương 6

System Validation and Implementation Evidence

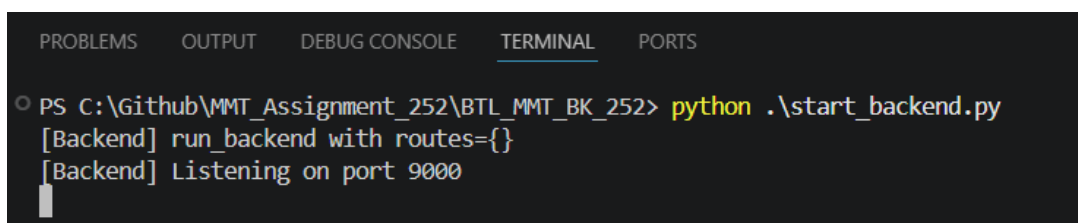
Chương này cung cấp các minh chứng thực tế nhằm xác thực tính đúng đắn của hệ thống so với các yêu cầu kỹ thuật đã đặt ra trong tài liệu đặc tả. Quá trình kiểm thử bao gồm việc khởi tạo các module độc lập của framework AsynapRous, khởi động hệ thống P2P, quy trình xác thực người dùng và vận hành các kịch bản Hybrid Chat.

6.1 AsynapRous Framework Initialization

Để kiểm chứng tính ổn định của các dịch vụ mạng nền tảng trước khi tích hợp, các thành phần cốt lõi của framework được khởi động và theo dõi độc lập thông qua terminal.

6.1.1 Backend Server Validation

Khi thực thi tập lệnh `start_backend.py`, tiến trình Backend được khởi tạo thành công và lắng nghe các kết nối TCP tới tại địa chỉ IP `0.0.0.0` trên cổng `9000`. Log trên terminal xác nhận hệ thống đã sẵn sàng tiếp nhận và xử lý đa luồng (Multi-threading) các yêu cầu từ client mà không bị gián đoạn (Non-blocking).



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Github\MMT_Assignment_252\BTL_MMT_BK_252> python .\start_backend.py
[Backend] run_backend with routes={}
[Backend] Listening on port 9000
```

Hình 6.1: Terminal log during Backend server initialization

6.1.2 Reverse Proxy Server Validation

Tiếp theo, hệ thống Proxy được khởi động thông qua tập lệnh `start_proxy.py`. Hệ thống ghi nhận Proxy đang lắng nghe tại cổng `8080`. Minh chứng cho thấy Proxy đã nạp thành công các quy tắc phân giải tên miền, sẵn sàng đóng vai trò trung gian chuyển tiếp (Reverse Proxy) các luồng traffic HTTP một cách an toàn.

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Github\MMT_Assignment_252\BTL_MMT_BK_252> python .\start_proxy.py
127.0.0.1:8080 ('127.0.0.1:9000', 'round-robin')
app1.local ('127.0.0.1:9001', 'round-robin')
app2.local (['127.0.0.1:9002', '127.0.0.1:9003'], 'round')
[Proxy] Listening on IP 0.0.0.0 port 8080
```

Hình 6.2: Terminal log during Proxy server initialization

6.1.3 Sample Web Application Validation

Để minh họa khả năng triển khai các endpoint RESTful, tập lệnh `start_sampleapp.py` được thực thi. Ứng dụng mẫu khởi chạy ổn định tại cổng 2026, cho thấy các decorator định tuyến (routing) của AsynapRous hoạt động chính xác trong việc ánh xạ URL với các phương thức HTTP (GET, POST, PUT).

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Github\MMT_Assignment_252\BTL_MMT_BK_252> python .\start_sampleapp.py
[Backend] run_backend with routes=({'POST', '/login': <function login at 0x00000242996F3420>, ('POST', '/echo'): <function echo at 0x000002429B95B7E0>, ('PUT', '/hello'): <function hello at 0x000002429B95BBA0>})
[Backend] Listening on port 2026
[Backend] route settings
+ ('POST', '/login'): <function login at 0x00000242996F3420>
+ ('POST', '/echo'): <function echo at 0x000002429B95B7E0>
+ ('PUT', '/hello'): **ASYNC** <function hello at 0x000002429B95BBA0>
```

Hình 6.3: Terminal log during Sample Application initialization

6.2 Hybrid Chat System Deployment

Hệ thống Chat P2P được điều phối tự động thông qua script `main.py`. Kịch bản này sử dụng thư viện `subprocess` để cấp phát các tiến trình độc lập. Như kết quả hiển thị trên Terminal, hệ thống đã khởi tạo thành công 3 tiến trình mạng chạy song song trên cùng một máy chủ (localhost) nhưng lắng nghe ở các cổng (port) khác nhau:

- **Tracker Server:** Khởi chạy tại cổng 9000.
- **Peer 1 Client:** Khởi chạy tại cổng 8001.
- **Peer 2 Client:** Khởi chạy tại cổng 8002.

```
PS C:\Github\MMT_Assignment_252\BTL_MMT_BK_252> python -c "from apps.tracker_app import create_tracker; create_tracker('0.0.0.0', 9000)"
[Backend] run_backend with routes=({'POST', '/submit-info': <function submit_info at 0x000001B75E5EA020>, ('POST', '/add-list': <function add_list at 0x000001B760B6E480>, ('POST', '/join-channel': <function join_channel at 0x000001B760B6E5C0>, ('GET', '/get-list': <function get_list at 0x000001B760B6E700>})
[Backend] Listening on port 9000
[Backend] route settings
+ ('POST', '/submit-info'): <function submit_info at 0x000001B75E5EA020>
+ ('POST', '/add-list'): <function add_list at 0x000001B760B6E480>
+ ('POST', '/join-channel'): <function join_channel at 0x000001B760B6E5C0>
+ ('GET', '/get-list'): <function get_list at 0x000001B760B6E700>

PS C:\Github\MMT_Assignment_252\BTL_MMT_BK_252> python -c "from apps.peer_app import create_peer; create_peer('0.0.0.0', 8001)"
[Backend] run_backend with routes=({'POST', '/login': <function login at 0x000001C64EB8EB60>, ('POST', '/connect-peer': <function connect_peer at 0x000001C64ECA1DA0>, ('POST', '/send-peer': <function send_peer at 0x000001C64ECA1EE0>, ('POST', '/broadcast-peer': <function broadcast_peer at 0x000001C64ECA2020>, ('POST', '/api/send-broadcast': <function api_send_broadcast at 0x000001C64ECA2160>, ('GET', '/get-messages': <function get_messages at 0x000001C64ECA22A0>})
[Backend] Listening on port 8001
[Backend] route settings
+ ('POST', '/login'): <function login at 0x000001C64EB8EB60>
+ ('POST', '/connect-peer'): <function connect_peer at 0x000001C64ECA1DA0>
+ ('POST', '/send-peer'): <function send_peer at 0x000001C64ECA1EE0>
+ ('POST', '/broadcast-peer'): <function broadcast_peer at 0x000001C64ECA2020>
+ ('POST', '/api/send-broadcast'): <function api_send_broadcast at 0x000001C64ECA2160>
+ ('GET', '/get-messages'): <function get_messages at 0x000001C64ECA22A0>

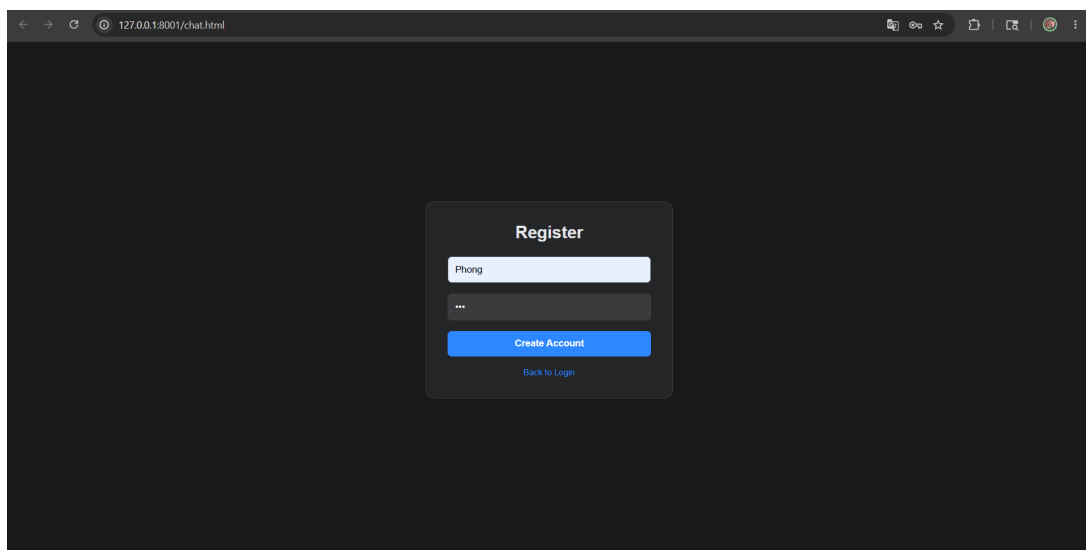
PS C:\Github\MMT_Assignment_252\BTL_MMT_BK_252> python -c "from apps.peer_app import create_peer; create_peer('0.0.0.0', 8002)"
[Backend] run_backend with routes=({'POST', '/login': <function login at 0x0000015888DBEB60>, ('POST', '/connect-peer': <function connect_peer at 0x0000015888DBEB1DA0>, ('POST', '/send-peer': <function send_peer at 0x0000015888DBEB1EE0>, ('POST', '/broadcast-peer': <function broadcast_peer at 0x0000015888DBEB2020>, ('POST', '/api/send-broadcast': <function api_send_broadcast at 0x0000015888DBEB2160>, ('GET', '/get-messages': <function get_messages at 0x0000015888DBEB22A0>})
[Backend] Listening on port 8002
[Backend] route settings
+ ('POST', '/login'): <function login at 0x0000015888DBEB60>
+ ('POST', '/connect-peer'): <function connect_peer at 0x0000015888DBEB1DA0>
+ ('POST', '/send-peer'): <function send_peer at 0x0000015888DBEB1EE0>
+ ('POST', '/broadcast-peer'): <function broadcast_peer at 0x0000015888DBEB2020>
+ ('POST', '/api/send-broadcast'): <function api_send_broadcast at 0x0000015888DBEB2160>
+ ('GET', '/get-messages'): <function get_messages at 0x0000015888DBEB22A0>
```

Hình 6.4: Terminal log during automated system initialization

6.3 User Registration and Authentication

6.3.1 User Registration (Đăng ký tài khoản)

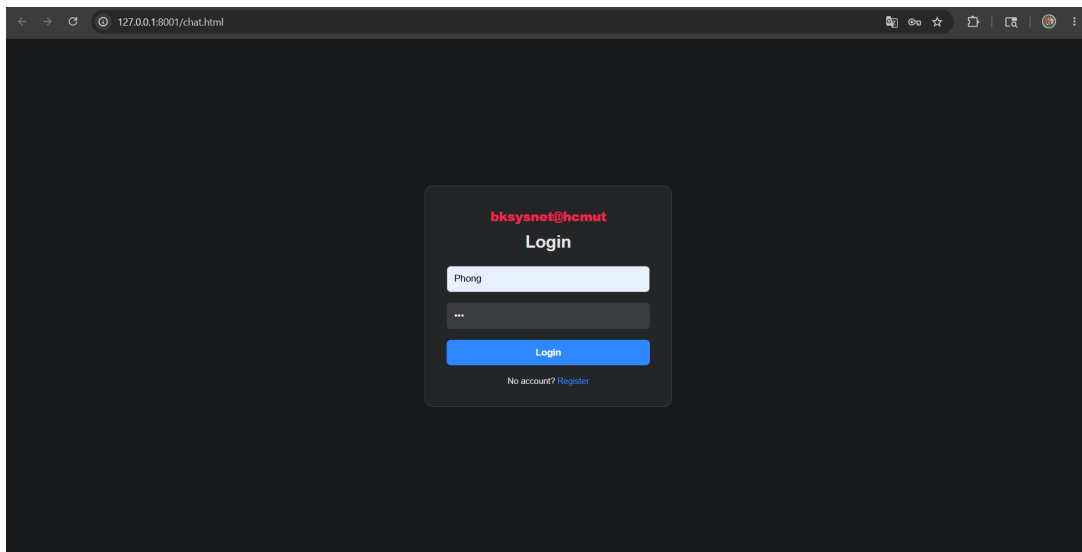
Để tham gia vào mạng lưới, người dùng mới bắt buộc phải khởi tạo tài khoản thông qua giao diện Đăng ký. Yêu cầu HTTP POST chứa thông tin được gửi đến endpoint `/register` của hệ thống. Máy chủ thực hiện kiểm tra tính duy nhất của username, và nếu hợp lệ, thông tin sẽ được lưu trữ cục bộ vào cơ sở dữ liệu (tệp JSON). Hệ thống phản hồi mã trạng thái thành công, cho phép người dùng lập tức sử dụng tài khoản này.



Hình 6.5: User Registration Form

6.3.2 User Login (Đăng nhập)

Chức năng xác thực được kiểm thử chặt chẽ thông qua giao diện Web. Người dùng truy cập vào trang và nhập thông tin đăng nhập. Dữ liệu được truyền tải qua endpoint `/login`. Khác với kiểm thử ban đầu, hệ thống sau khi đối chiếu thông tin hợp lệ sẽ không chỉ trả về thông báo thông thường mà tiến hành lưu phiên làm việc (session/cookie), đồng thời thông báo cho Tracker biết trạng thái trực tuyến (online) của người dùng trước khi điều hướng họ thẳng vào Chat Dashboard.



Hình 6.6: User Authentication Form (`login.html`)

6.4 Hybrid Chat Application: End-to-End P2P Validation

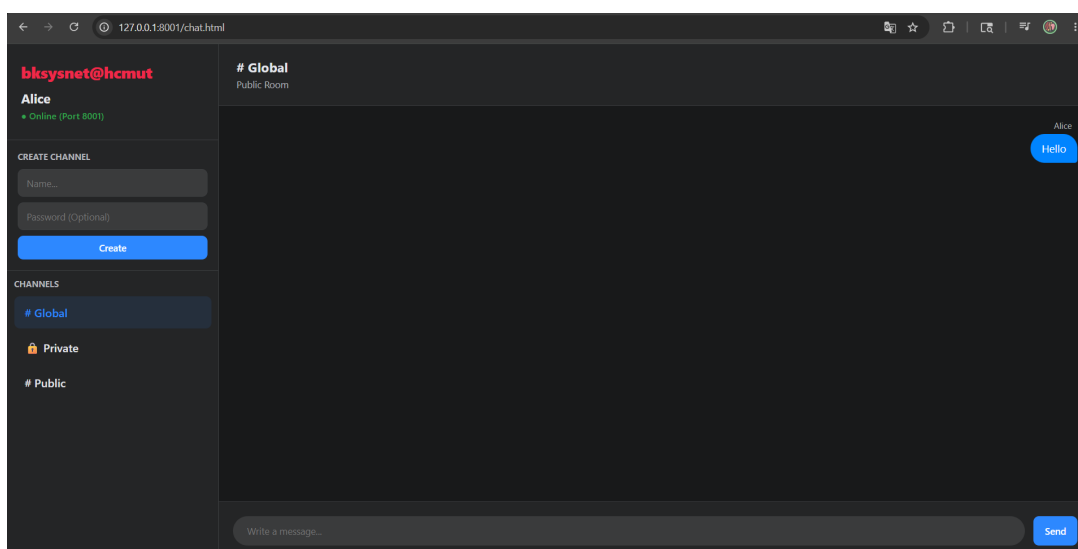
Sau khi đăng nhập và xác thực thành công, hai người dùng (Alice tại port 8001 và Bob tại port 8002) được chuyển hướng tới giao diện Chat Dashboard (`chat.html`). Dưới đây là các kịch bản kiểm thử nhằm đánh giá toàn diện năng lực của kiến trúc Hybrid P2P.

6.4.1 Nhắn tin Broadcast trong kênh Global

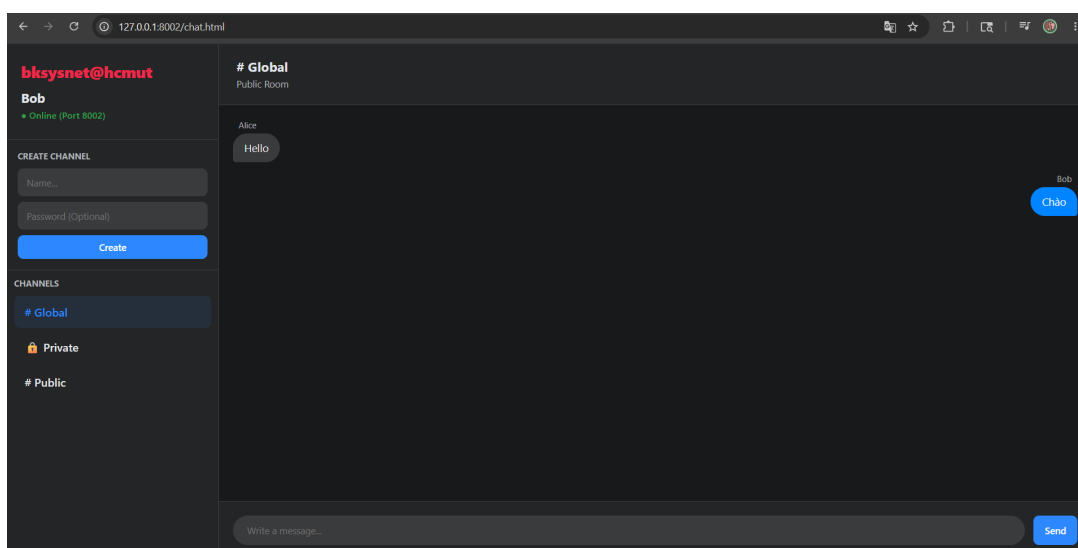
Quá trình kiểm thử tính năng phát sóng (Fan-out) trực tiếp giữa các Peer diễn ra như sau:

1. User Alice nhập nội dung *"Hello"* vào kênh Global và nhấn Send.
2. Tại cửa sổ của user Alice (Hình 6.7), tin nhắn ngay lập tức hiển thị.
3. User Bob phản hồi *"Chào"*, và nội dung lập tức xuất hiện trên màn hình của Bob (Hình 6.8).

Sự xuất hiện tức thời của các bong bóng chat chứng tỏ cơ chế "Client-Side Fan-out" và endpoint `/broadcast-peer` đang hoạt động trơn tru. Hệ thống P2P đã mở port tính thành công, thiết lập kết nối chéo giữa các Peer để truyền tải dữ liệu trực tiếp mà không bị gián đoạn hay tắc nghẽn I/O tại máy chủ trung tâm.



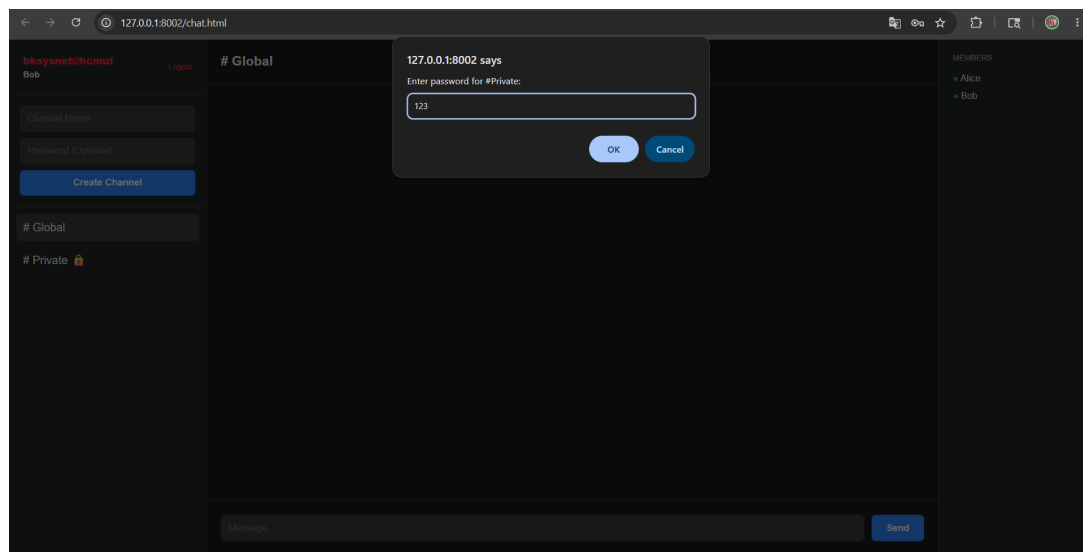
Hình 6.7: Chat UI for Peer 1 (User: Alice, Port: 8001) showing broadcast capability



Hình 6.8: Chat UI for Peer 2 (User: Bob, Port: 8002) successfully receiving P2P messages

6.4.2 Quản lý và tham gia kênh Chat riêng tư (Private Channel)

Trong kịch bản này, Alice thực hiện tạo một kênh mới có đặt mật khẩu bảo vệ để làm việc nhóm. Tracker nhận thông tin khởi tạo và phân phối danh sách kênh mới cho toàn mạng. Khi Bob nhấp vào kênh này, hệ thống sẽ yêu cầu nhập mật khẩu xác thực. Chỉ khi Bob nhập đúng mật khẩu, anh mới được Tracker phê duyệt đưa vào danh sách thành viên hợp lệ của kênh (members array) để bắt đầu gửi và nhận tin nhắn P2P cục bộ trong kênh đó.



Hình 6.9: Authentication process for joining a private P2P channel

Chương 7

Conclusion

7.1 Summary of Achievements

Nhìn lại toàn bộ quá trình nghiên cứu và thực hiện Assignment 1, nhóm chúng em đã hoàn thành xuất sắc các mục tiêu và yêu cầu đề ra. Không phụ thuộc vào bất kỳ framework web có sẵn nào (như Flask, Django hay Express), nhóm đã tự xây dựng thành công framework **AsynapRous** hoàn toàn từ con số không, hoạt động trực tiếp trên nền tảng Socket API thô của hệ điều hành.

Hệ thống đã hiện thực hóa thành công 3 trụ cột kỹ thuật lõi:

1. **Non-blocking HTTP Server:** Thiết kế kiến trúc máy chủ linh hoạt hỗ trợ nhiều mô hình I/O (Multi-threading, Event-driven qua `selectors`, và Coroutine qua `asyncio`). Việc xử lý bất đồng bộ giúp hệ thống chịu tải tốt, không bị nghẽn (bottleneck) khi có nhiều kết nối đồng thời.
2. **Secure Session Management:** Triển khai quy trình kiểm soát truy cập (Access Control) tuân thủ chặt chẽ các RFC tiêu chuẩn của HTTP. Hệ thống đã tự động bóc tách và giải mã Basic Auth (RFC 2617), đồng thời thiết lập và quản lý vòng đời của phiên làm việc qua thẻ `Set-Cookie` với các cờ bảo mật như `HttpOnly`.
3. **Hybrid P2P Chat Application:** Xây dựng một mạng lưới phân tán thông minh. Hệ thống phân tách rõ ràng giữa "Control Plane" (Tracker Server lo việc định tuyến, lưu danh bạ và cấp quyền) và "Data Plane" (các Peer tự thiết lập TCP Socket để truyền tin trực tiếp).

7.2 Technical Reflections and Learnings

Việc phải tự tay "mổ xẻ" và xử lý từng byte dữ liệu ở tầng giao vận (Transport Layer) đã mang lại những bài học kỹ thuật vô giá, vượt xa việc chỉ sử dụng các API bậc cao:

- **Bản chất của giao thức HTTP:** Chúng em đã hiểu sâu sắc rằng HTTP thực chất chỉ là một giao thức văn bản (text-based protocol) chạy trên nền TCP. Việc tự viết các hàm phân tích chuỗi (parse headers), trích xuất boundary của multipart form, hay thiết lập `Content-Length` đã giúp nhóm thấu hiểu vòng đời của một gói tin Request/Response.

- **Giải quyết bài toán Concurrency:** Xử lý đa luồng (multi-threading) trên Tracker Server đòi hỏi sự cẩn trọng với các vấn đề Race Condition. Việc áp dụng các cơ chế khóa (Locks) khi đọc/ghi vào biến bộ nhớ chung `active_channels` là bài học thực tế về lập trình song song. Bên cạnh đó, việc cấu hình `socket.setblocking(False)` đã cho thấy sức mạnh của mô hình Event-driven so với Thread-per-request truyền thống.
- **Tư duy thiết kế hệ thống phân tán (Distributed System Design):** Thuật toán "Client-Side Fan-out" trong chức năng Broadcast là một sự đánh đổi (trade-off) kiến trúc tuyệt vời. Thay vì dồn mọi băng thông vào một máy chủ trung tâm (gây ra điểm mù Single Point of Failure), việc ép các Client tự phân phối tin nhắn đã chuyển bài toán mở rộng (scalability) sang mạng ngang hàng, giảm triệt để tải cho Tracker.

7.3 Closing Thoughts

Đồ án **Assignment 1** là một thử thách lớn, đòi hỏi sự kiên nhẫn trong việc debug ở cấp độ mạng thấp nhất. Tuy nhiên, chính việc phải đối mặt với các lỗi như **Broken Pipe**, **Connection Refused**, hay **Socket Timeout** đã rèn luyện cho nhóm một tư duy hệ thống sắc bén.

Thành quả đạt được không chỉ là một framework có thể chạy được, mà là sự tự tin trong việc thiết kế, vận hành và làm chủ các kiến trúc mạng máy tính hiện đại. Những kiến thức nền tảng vững chắc này sẽ là hành trang quý giá cho chúng em trên con đường trở thành những Kỹ sư thực thụ.